Editorial

Tasks, test data and solutions were prepared by: Vito Anić, Fran Babić, Toni Brajko, Martina Licul, Petar Sruk, and Iva Tadić.

Implementation examples are given in attached source code files.

Task Zlagalica

Prepared by: Iva Tadić
Necessary skills: for loop, matrices

When solving this task, key is to notice that by connecting pieces, the entire puzzle can only expand upwards and to the right. Since the constraints are small enough, we know that in the worst case scenario (when we have 20 puzzles with a height and width of 10), our completed puzzle cannot have more than 200 rows and 200 columns. Because of this, and the fact that the puzzle can only "expand" upwards and to the right, we will create a solution matrix with at least that many rows and columns, and we will start assembling our puzzle from its bottom-left point.

Now, in the *for loop*, we follow the order of connecting given in the task. Each time after adding a puzzle piece to the matrix, we calculate from which point (x, y) in the matrix will we start building the next piece (we always start building each piece from its bottom-left corner and then go upwards and to the right). Let (a, b) be the point in the matrix where the bottom-left corner of the puzzle piece we just placed in the matrix is located, let r and s be the number of its rows and columns, and u and d be the numbers written on its back. If the next puzzle is added on top of the previous one (i.e., $u = 0$), (x, y) will be equal to $(a - r, b + d - 1)$. If the next puzzle is added to the right of the previous one (i.e., $u = 1$), (x, y) will be equal to $(a - r + d, b + s)$.

To know the final height and width of the assembled puzzle, we keep track of the "highest" row and the "rightmost" column we have reached while assembling. Using them, the dimensions of the finished puzzle can be easily calculated, as well as the requested image printed.

Task Bitovi

Prepared by: Toni Brajko
Necessary skills: constructive algorithms, bitmasks

In the first subtask, the highest weight bit (if we consider all numbers as 15-bit numbers) is always 0. To transform one number into another through a sequence of moves, it is enough to set the highest bit to 1, arbitrarily change the other bits, and then revert the highest bit to 0. However, we must ensure that the number we obtain is not currently in set A.

In the second subtask, there will always exist a path (a sequence of moves) to transform any number into any other number, without changing the rest. Moreover, as long as $|A| \leq b$, where b is the number of bits in the representation of each number, there exists a path from x to y for $x \in A$ and $y \notin A$. Now, it is sufficient to find an arbitrary pair (x, y) such that $x \in A \setminus B$ and $y \in B \setminus A$, and then change x to y. If such a pair does not exist, sets A and B are equal.

We solve the last two subtasks similarly. Let's consider an arbitrary pair (x, y) that satisfies the above condition and an arbitrary path from x to y without the restriction that it cannot contain elements from set A. Let z be an element from A on that path closest to the number y. Then, on the path from z to y, there are no elements from set A. Therefore, we can change z to y through a sequence of moves, and then recursively repeat this process for (x, z) until we obtain a simple path from the previous subtask. Finding all required variables is done in $\mathcal{O}(N)$, the maximum possible length of the path is $\mathcal{O}(N \cdot b)$, and there are at most $\mathcal{O}(N)$ such pairs, so the overall complexity of this algorithm is $\mathcal{O}(N^2 \cdot b)$, which is sufficient to solve the third subtask.

Since the choice of the path is arbitrary, let's choose the shortest path whose length is not greater than the number of bits b. Then, the sum of the lengths of all paths is $\mathcal{O}(N \cdot b)$. Note that after executing the algorithm for (x, y), we will have $x \in B$, and every number that was in both A and B before executing the algorithm will still be in both sets. Therefore, we can iterate over the elements of set B, and for each number $y \in B$ that is not in A, we call the algorithm, using any $x \in A$ that is not in B. Such x's can be maintained in various ways, and in the provided source code, it is implemented using a `set`. The overall complexity is $\mathcal{O}(N \cdot b + N \log N)$.

Task Piratski kod

Prepared by: Petar Sruk
Necessary skills: Dynamic programming

For 20 points it was enough to carefully implement a brute force algorithm calculating values of all pirate codes.

Let's introduce some notation:
$C(k)$ - number of pirate entries of length k.
$S(k)$ - sum of values of all pirate entries of length k.
$f(n)$ - sum of values of all pirate codes of length n.

We can easily notice that $f(n) = \sum_{k=1}^{n} C(k) \cdot f(n-k) + 2^{n-k} \cdot S(k)$. It only remains to determine $C(k)$ i $S(k)$.

Number of pirate entries of length k is $C(k) = \text{Fib}[k-1]$ because we see that from a pirate entry of length $k-2$, by changing the last digit to 0 and concatenating two ones at the end we get a pirate entry of length k that ends with 1011. From a pirate entry of length $k-1$, by changing second to last digit to 0 and concatenating a single 1 at the end, we get a pirate entry of length k that ends with 0011. From this analysis we get a recursion of fibonacci numbers and we can easily check the basis for $k = 1, 2$.

Pirate entries of length k represent numbers from $\text{Fib}[k]$ to $\text{Fib}[k+1] - 1$ so the following holds:

$$S(k) = \frac{1}{2}(\text{Fib}[k+1](\text{Fib}[k+1] - 1) - \text{Fib}[k](\text{Fib}[k] + 1)).$$

Now we can finish the task with dynamic programming in time complexity $O(n^2)$.

Task Rolete

Prepared by: Vito Anić
Necessary skills: ad hoc, greedy or alternatively ternary search and binary search

To solve the first subtask, it was sufficient to fix each possible number of parallel blind lifts for each query, and then by traversing the blinds, determine how many manual blind lifts are needed. Time complexity: $O(n \cdot q \cdot maxa)$.

Subtask $q = 1$ could be solved in a similar way. Instead of traversing the blinds to determine the number of manual lifts, we will sort the blinds in descending order at the beginning. Now we can notice that some prefix of the blinds will always need to be manually lifted. We can find exactly which prefix by using binary search. To determine how much time will be needed, we will use prefix sums. Time complexity: $O(q \cdot maxa \cdot log(n))$.

The entire task could be solved in two completely different ways. The first method, which we won't go into depth in here, is to notice that the function of cost depending on the number of parallel lifts is convex. By using ternary search, we can determine the most efficient solution for each query.

For the second method, we will compute the solution for every possible blind height. Let's assume that for a certain height h, we have calculated the fastest way to lower all blinds so that all are lowered by at most

h centimeters. Let the blinds then be in the state $b_1, b_2, ..., b_n$. It obviously holds that $b_1, b_2, ..., b_n \leq h$. We want to go from this state to a state that has max height of $h-1$. To do this we will perform one of the following two operations:

- We will raise all blinds parallel by 1 centimeter, thus reaching a state where the maximum height is $h-1$.
- We will manually lower all blinds with a height of h by 1 centimeter.

Choosing the operation that takes less time, it holds that this will be the best way to reach the state $h-1$. Proof is left as an exercise for the reader.

If we start from the highest possible height, we can build solutions towards smaller heights. For details on implementation, look at the code provided in the attachment. Time complexity: $O(maxa)$.

Task Trokut

Prepared by: Fran Babić
Necessary skills: Game theory, Sprague-Grundy theorem

For 13 points, it was sufficient to recursively determine whether a game state is winning or losing. However, it is necessary to optimize the solution using memoization technique.

To solve the second subtask worth 36 points, we need to notice the following: the player who uses a point that is already connected to another point by a line segment in their move will lose in the next move. Note that this is also a necessary condition for a player to lose. Therefore, we can play a similar and equivalent game to the previous one: a player on their turn connects two line segments so that no two line segments intersect (or touch), and the loser is the player who can no longer make a move.

Let's consider the initial state of the game for some N. When we connect two points with a line segment, we divide the game into two independent "boards" of sizes l and r such that $l + r = N - 2$. Now we can calculate the Grundy number of each state, i.e., for each N using the previous ones using the following equation:

$$g[n] = \text{mex}\{g[l] \oplus g[r] : l + r = n - 2\}$$

where mex$\{\ldots\}$ denotes *minimum excludant value*. It is now obvious that Lucija will win for some N if and only if $g[N] > 0$. The complexity of this algorithm is $O(N^2)$.

To achieve the maximum number of points on the task, it was necessary to notice that the sequence $g[n]$ enters a period of length 34 after a relatively small pre-period, which allows us to efficiently determine the Grundy numbers for any $N \leq 10^9$. We did not prove some of the claims and leave them as the exercise to the reader. Implementation details can be found in the official source code.